

OBLIG 2- Teori

Teorioppgave 1 – if-test/if-statement

Forklar hva en if-test/if-statement er med egne ord.

If-test/if-statement er en kode struktur som brukes i programmering for å få et program til å ta forskjellige avgjørelser basert på if/else oppsettet og innkommende data som blir knyttet til if/else vurderingen.

Eksempelvis så kan en if/else kode se ut som dette:

```
user = Input("name")  
  
if (user == "Andreas"):  
    print("Velkommen inn Andreas. ")  
  
else:  
    print("Du er ikke Andreas!")
```

Her har vi en if/else statement som vurderer innholdet på variabelen user. Om string innholdet i variabelen er «Andreas» vil den gi beskjeden «Velkommen inn Andreas.», men om variabelen inneholder noe annet en dette vil den i stede skrive ut «Du er ikke Andreas!»

Beskriv også sammenhengen mellom if, elif og else.

Disse tre uttrykkene er en sjekkordning med hierarki rekkefølge, hvor programmet sjekker i rekkefølge om data passer i forhold til de kravene som er blitt spesifisert i uttrykket. If uttrykket er alltid den som blir vurdert først. Om kravet blir oppfylt blir koden som er knyttet til if kjørt. Om den ikke stemmer vil programmet gå videre til å sjekke om kravet blir oppfylt i noen av elif uttrykkene (om ja så kjører den koden, ellers går den videre). Elif uttrykkene blir sjekket i den rekkefølgen de er satt opp. Om ingen av elif uttrykkene skulle innfri kravene, vil den gå til siste løsning som er else. Else er det gjerne uttrykket som blir brukt om ingen av kravene i if eller elif uttrykkene blir oppfylt.

Eksempel:

```
nummer = input(int("Skriv et tall mellom 4 og 6"))  
  
if (nummer >= 4):  
    print(f "Du valgte: {nummer}")  
  
elif (nummer <= 6):  
    print(f "Du valgte: {nummer}")
```

else:

```
print("Det er ikke hva vi ba deg skrive»")
```

Teorioppgave 2 - liste

Forklar hva lister er i Python og hva de kan brukes til.

Lister er variabler som kan holde på flere verdier/elementer. Dette vil si at du kan få en variabel til å holde på mer enn en verdi av gangen ved å bruke lister. Lister kan også blande forskjellige typer datasett i en liste, og kan også inneholde flere lister som blir nestet i listen. Lister brukes som et organiseringsverktøy som gjør kodestruktur mer oversiktlig og brukervennlig. I stedet for at vi lager en separat variabel for hver enkelt verdi eller element i koden, så kan vi koble alt til en enkelt variabel og hente ut de ønskede elementene ved å referere til elementets plassering i listen.

Eksempel:

```
ting = ['bil', 'båt', 14, 'katt']
```

```
print (ting[1])
```

Denne koden vil printe ut string verdien "båt"

Teorioppgave 3 - løkker

Forklar hva "løkker" er, og gi noen eksempler på tilfeller man kan ønske å benytte dem.

Løkker er en funksjon i programmering for å [iterere](#) en kode. Denne løsningen forenkler måten vi koder på ved at vi får muligheten til å kjøre ut flere uttrykk fra en linje med kode fremfor at vi må hente ut data med et uttrykk av gangen.

I hovedsak har vi tre typer løkker:

For – Blir brukt for å iterere en satt sekvens ([Web forelesning](#)).

"for each" – Brukes for å iterere elementer i en liste eller sekvenser i en string ([Web forelesning](#)).

While – En betingelse definert løkke som repeterer koden frem til betingelsen ikke lenger blir oppfylt ([Web forelesning](#)).

Et eksempel på «for each» er om du ønsker å printe ut en oversikt over hvilke elementer som finnes i en liste. I stedet for at du da må definere hvert enkelt element i listen, kan du sette en løkke til å printe ut listen for deg.

Eksempel:

```
ting = ['bil', 'båt', 14, 'katt']
```

for liste in ting:

```
print (liste)
```

I eksempelet over vil *for each* løkken først definere variabelen *liste* som det første elementet i listen til variabelen *ting* og deretter printe dette. Så vil den gå til det neste elementet i listen å redefinere variabelen *liste* til den gitte verdien, for så å printe dette. Denne prosessen blir gjentatt til listens slutt. Variabelen *liste* ha verdien til det siste elementet i listen når løkken er ferdig.

En annen funksjon som løkker kan brukes til er blant annet å telle på forskjellige måter(*for*). Man kan definere hvilken verdi den skal starte og slutte å telle på, samt hvilken måte den skal tekke på (2 gangen, 3 gangen, 17 gangen, osv.)

Eksempel:

```
oddNumbers = [odd for odd in range(1, 21, 2)]
```

```
print(oddNumbers)
```

Denne koden vil telle fra 1 til (ikke til og med) 21 med 2 gangen. Den vil dermed gi ut verdiene [1, 3, 5, 7, 9, 11, 13, 15, 17, 19] i form av en liste.

Til slutt har vi et eksempel på en while løkke. Denne koden under vil gi deg en gåte, og frem til du svarer riktig vil den spørre deg igjen og igjen:

```
correct = False
```

```
while not correct:
```

```
    answer = input("Hva går og går, men aldri kommer til døra?\n")
```

```
    answer = answer.title()
```

```
    print(answer)
```

```
    if answer == "Klokka":
```

```
        print("riktig!")
```

```
        correct = True
```

```
    else:
```

```
        print("Prøv igjen\n")
```

Kilder	
Forelesning	Nils-Christian Walthinsen Rabben
Python Crash Course 3'rd edition	Pensum book
Hiof youtube instruksjonsvideoer	https://www.youtube.com/watch?v=s2qm7wexoAY&t=2s&ab_channel=Hi%C3%98-InformasjonteknologiogKommunikasjon